GIS算法设计与分析

第9次课 · 地图渲染与可视化引擎

把数据变成用户能看懂的地图——G8组核心任务课

矢量瓦片 · 渲染管线 · 样式系统 · 文字标注 · 性能优化 · UAV/AUV可视化

授课对象：大三本科生 | 授课时长：2小时

课程回顾

前8次课脉络与本节课定位

前8次课回顾

• 第1-2次课：导航系统全景与地图数据生产• 第3-4次课：路网模型、拓扑构建与空间索引• 第5-6次课：路径规划引擎与实时定位匹配• 第7-8次课：导航引擎状态机与离线存储• ——数据层、算法层、引擎层已逐步建立

本节课定位——G8渲染引擎组

• G8组核心任务：地图渲染与可视化引擎• 承上：消费G1-G7产出的地图数据与导航状态• 启下：为G10语音交互组提供可视化界面• 核心挑战：海量数据 + 实时渲染 + 多端适配• 关键产出：可运行的地图渲染引擎原型

学习目标

知识目标·能力目标·工程目标

01

理解地图渲染五层架构与渲染管线全流程

02

掌握矢量瓦片解析、样式系统与文字标注核心算法

03

掌握性能优化关键技术：Draw Call、LRU缓存、视锥体剔除

04

了解无人机/水下AUV等特种可视化场景

💡 **带着这些问题，我们开始今天的内容**

核心算法

本节课涉及的17个核心算法（按重要度分级）

★★★	**矢量瓦片解析与LOD调度**	MVT格式解析、瓦片金字塔、多细节层次调度策略，渲染引擎数据入口的核心
★★★	**样式表达式求值引擎**	Mapbox Style Specification表达式解析与运行时求值，支持数据驱动样式
★★★	**文字碰撞检测与标注放置**	点/线/面标注算法、贪心碰撞检测、优先级队列，决定地图可读性
★★★	**SDF/MSDF文字渲染**	有向距离场文字渲染、多通道SDF，实现任意缩放的清晰文字效果
★★★	**视锥体剔除与批量渲染**	视锥体裁剪、Draw Call合并、GPU批量提交，性能优化的核心手段
★★☆	**墨卡托投影与坐标变换**	Web墨卡托投影公式、瓦片坐标系统、经纬度↔像素坐标转换

移动地图与组成要素：

主流地图形态和要素---百度、高德、eCharts、Mapbox









核心算法--移动地图与组成要素





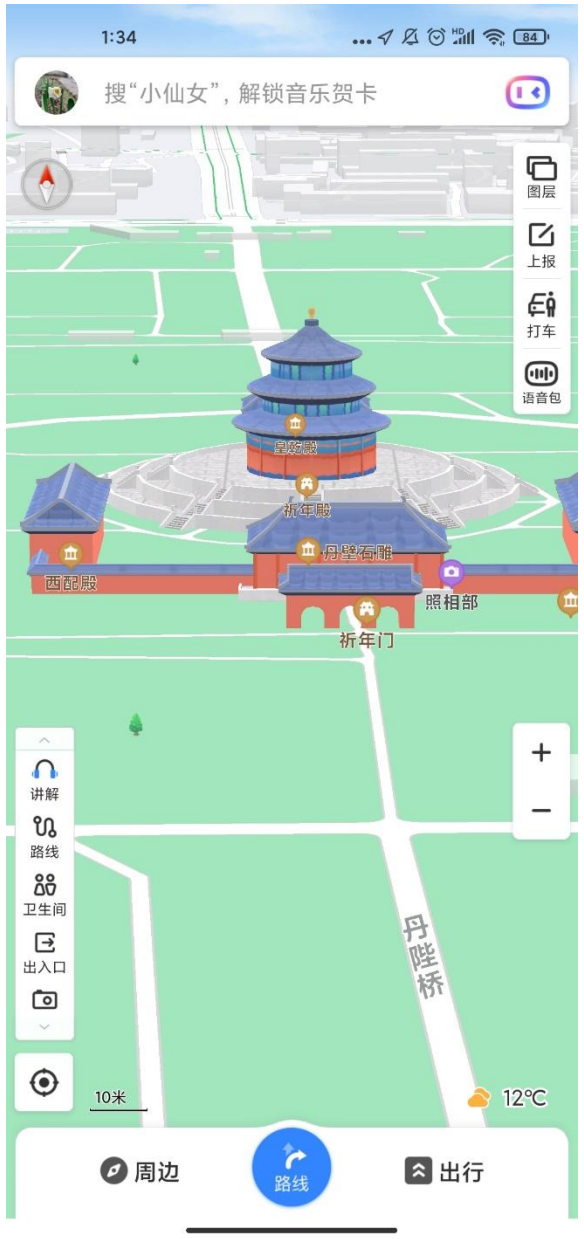




附加地图要素：

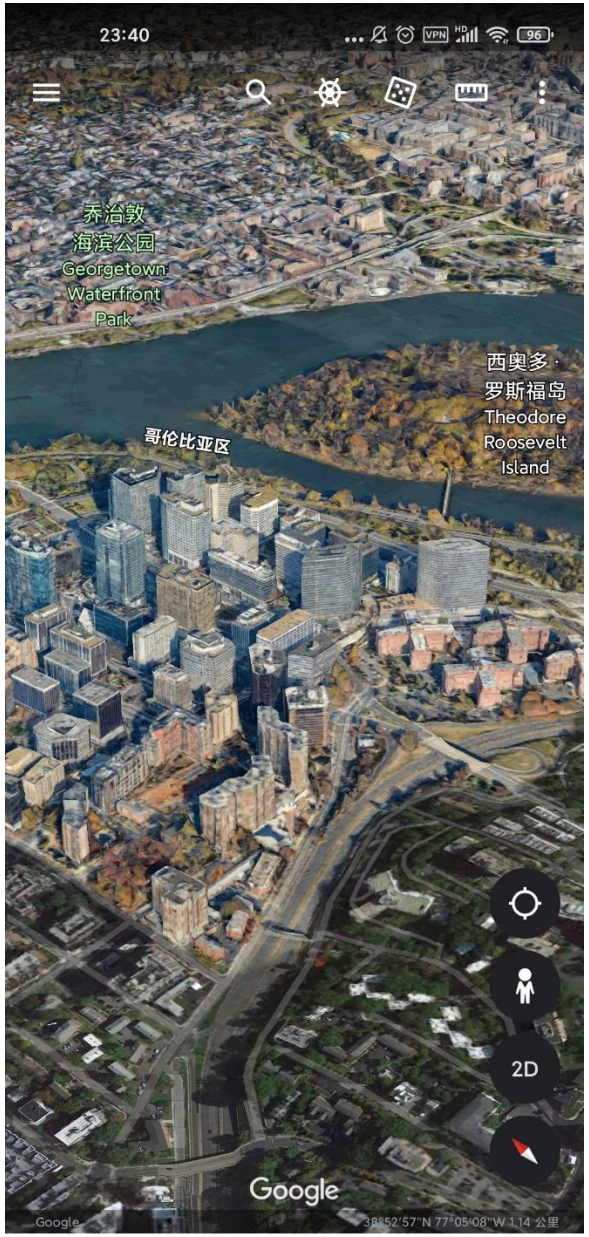
三维模型：人工矢量建模或点云模型

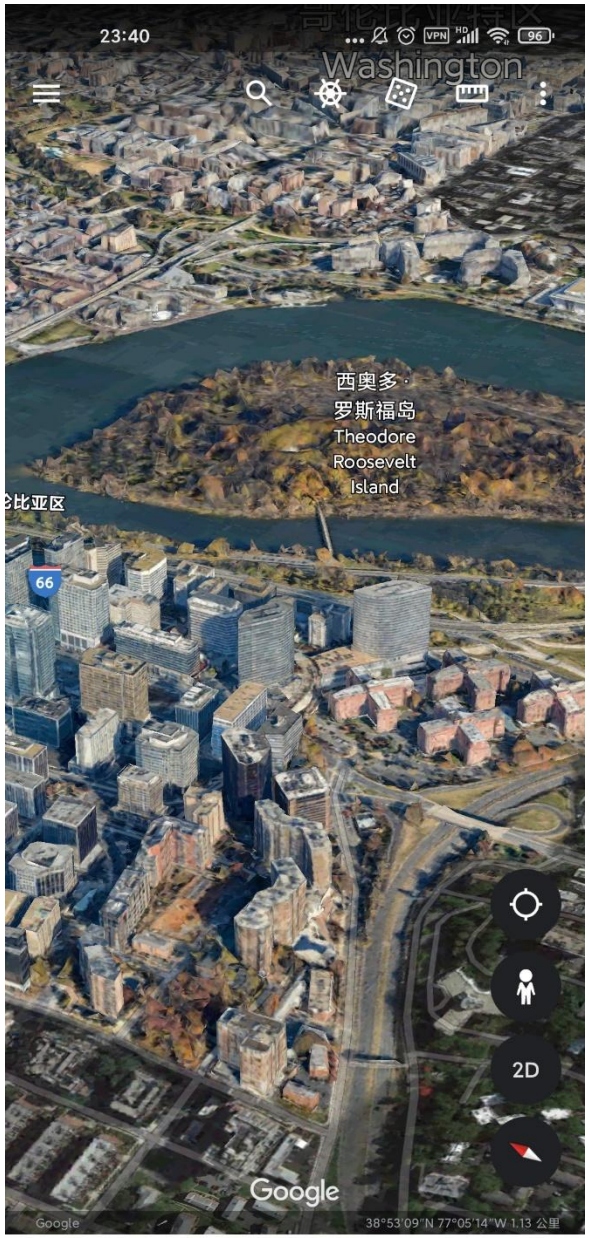
卫星影像：栅格数据

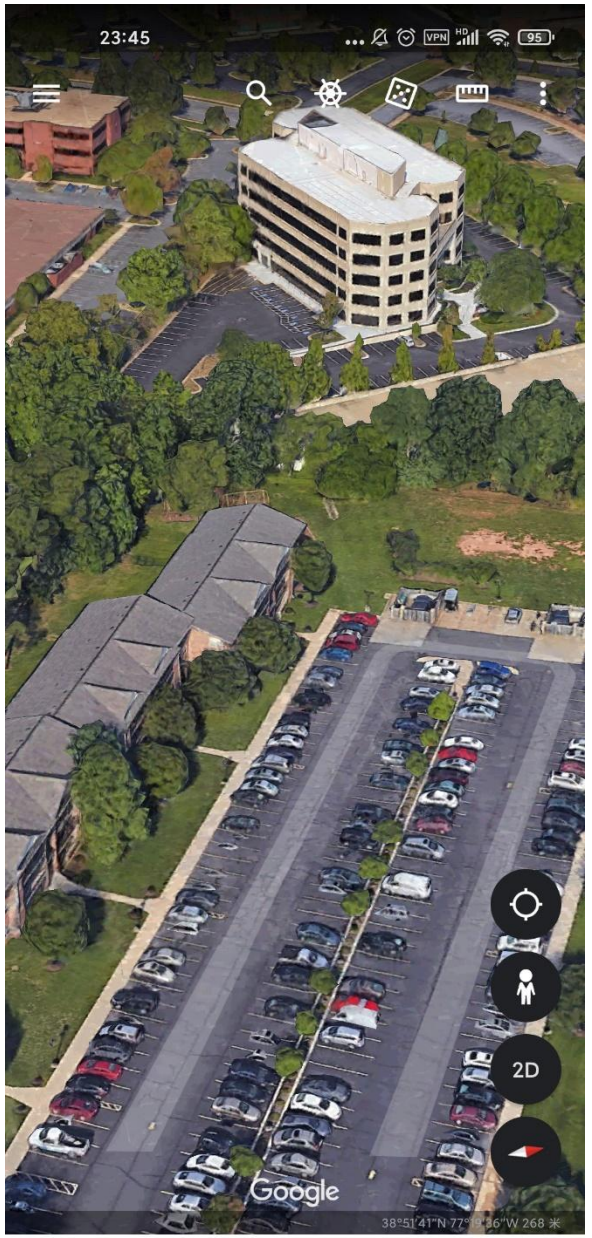


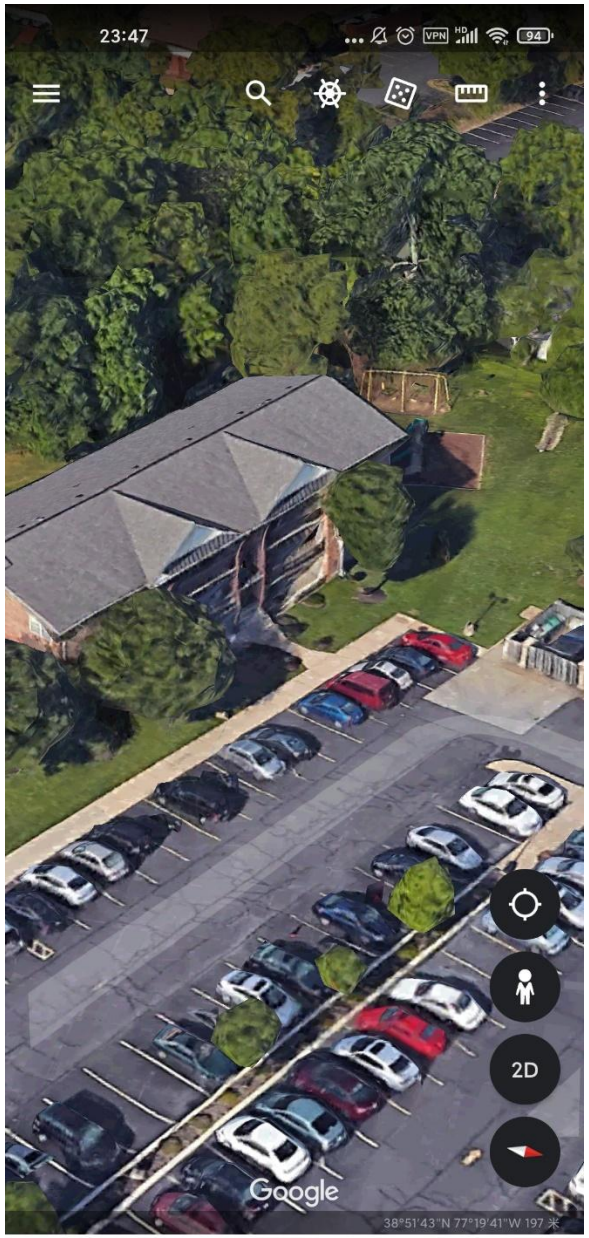


核心算法--移动地图与组成要素









CHAPTER · 核心问题

为什么地图加载快、看起来还这么好看？

- 海量数据如何在手机上60帧流畅渲染？
- 文字标注为什么从不重叠、永远清晰？
- 一套代码如何同时跑在Web/移动端/桌面端？

核心挑战

地图渲染引擎面临的四大核心挑战

数据量 vs 实时渲染

全国道路+POI达GB级，需在60fps下实时渲染，单帧处理百万级要素

多尺度缩放表达

从全球概览到街道级，18级缩放跨度，每级都要有合适的信息密度

文字标注质量

数千条标注同时存在，需解决碰撞、避让、优先级，保证可读性

跨平台适配

WebGL/Metal/Vulkan多后端，PC/手机/车机多设备，性能差异巨大

地图渲染引擎的五层架构

L5	应用接口层	对外提供地图API：地图控件、标注管理、事件监听、图层控制、样式切换接口
L4	业务逻辑层	标注布局、交互控制、动画系统、样式管理、图层调度、视图状态管理
L3	核心渲染层	渲染管线、着色器管理、纹理系统、顶点缓冲、Draw Call管理、GPU资源池
L2	渲染抽象层	Rendering Abstraction Layer：统一WebGL/Metal/Vulkan/D3D，封装着色器、纹理、缓冲
L1	平台层	操作系统与图形API：浏览器WebGL、iOS Metal、Android OpenGL ES/Vulkan

架构总览

地图渲染引擎五层架构——分层解耦

上层：面向应用与用户

L5 **应用接口层**

地图控件 · 标注管理 · 事件监听 · 图层控制 · 样式切换接口

L4 **业务逻辑层**

标注布局 · 交互控制 · 动画系统 · 样式管理 · 图层调度 · 视图状态

L3 **核心渲染层**

渲染管线 · 着色器管理 · 纹理系统 · 顶点缓冲 · Draw Call管理 · GPU资源池

L2 **渲染抽象层**

统一 WebGL / Metal / Vulkan / D3D，封装着色器、纹理、缓冲（RAL）

L1 **平台层**

浏览器 WebGL · iOS Metal · Android OpenGL ES / Vulkan

▲ 调用方向：上层依赖下层；L2 渲染抽象层是跨平台的关键——一套上层代码适配多种图形API

设计要点

- 层间只经接口通信
- L3 管理GPU资源池
- L2 屏蔽平台差异
- L1 由操作系统提供
- 每层可独立替换

G8 渲染引擎组

的自研边界主要在 L3 与 L4 两层

渲染管线

地图渲染的五个核心阶段

Step 1

数据加载

瓦片请求、网络/缓存读取、MVT/PBF解码

Step 2

数据解析

几何解析、属性提取、坐标变换、投影转换

Step 3

样式计算

表达式求值、图层过滤、颜色/线宽计算、分类

Step 4

几何构建

三角剖分、线转条带、文字轮廓生成、顶点缓冲

Step 5

渲染提交

状态设置、Draw Call、批量提交、后处理、帧缓冲

60fps	16ms	5阶段	~1M
目标帧率	单帧预算	管线划分	单帧要素

渲染管线

地图渲染管线五阶段——一帧16ms的预算

1 数据加载

瓦片请求 · 网络/缓存读取 · MVT/PBF 解码

2 数据解析

几何解析 · 属性提取 · 坐标变换 · 投影转换

3 样式计算

表达式求值 · 图层过滤 · 颜色/线宽计算

4 几何构建

三角剖分 · 线转条带 · 文字轮廓 · 顶点缓冲

5 渲染提交

状态设置 · Draw Call · 批量提交 · 后处理

每帧循环：请求下一批瓦片 → 解析 → 求值 → 构建 → 提交（60fps = 单帧 16ms 预算）

单帧时间预算（示意）



批处理合并 Draw Call

同类要素合并为一个批次，Draw Call 数量决定CPU开销

瓦片解码放后台线程

主线程只做样式求值与提交，避免解码阻塞渲染

超预算即掉帧

16ms内未完成 → 帧率下降，优先保证视口内要素

关键指标：单帧 ~1M 要素 · 5阶段 · 60fps —— 每一阶段的耗时都要在预算内

MVT格式详解——Mapbox Vector Tile

MVT格式结构

• 基于Protocol Buffers的二进制格式• 一个瓦片包含多个图层（layer）• 每个图层包含要素（feature）集合• 要素类型：点/线/面三类• 几何采用命令式编码（Command Integers）• 坐标使用瓦片内相对整数（0-4095）• 属性表去重存储，节省空间 MoveTo / LineTo / ClosePath 命令 整数Delta编码压缩坐标差值

矢量瓦片 vs 栅格瓦片

• 体积小：矢量瓦片约5-50KB/片• 样式可定制：客户端动态渲染样式• 缩放无损：任意缩放保持清晰• 支持交互：要素可点击、可查询• 多语言切换：无需重新生成瓦片• 客户端计算量大，对GPU要求高• 栅格瓦片：服务器渲染，客户端简单• 矢量瓦片：数据+样式分离，灵活度高• MVT已成为行业事实标准

瓦片金字塔模型与LOD细节层次

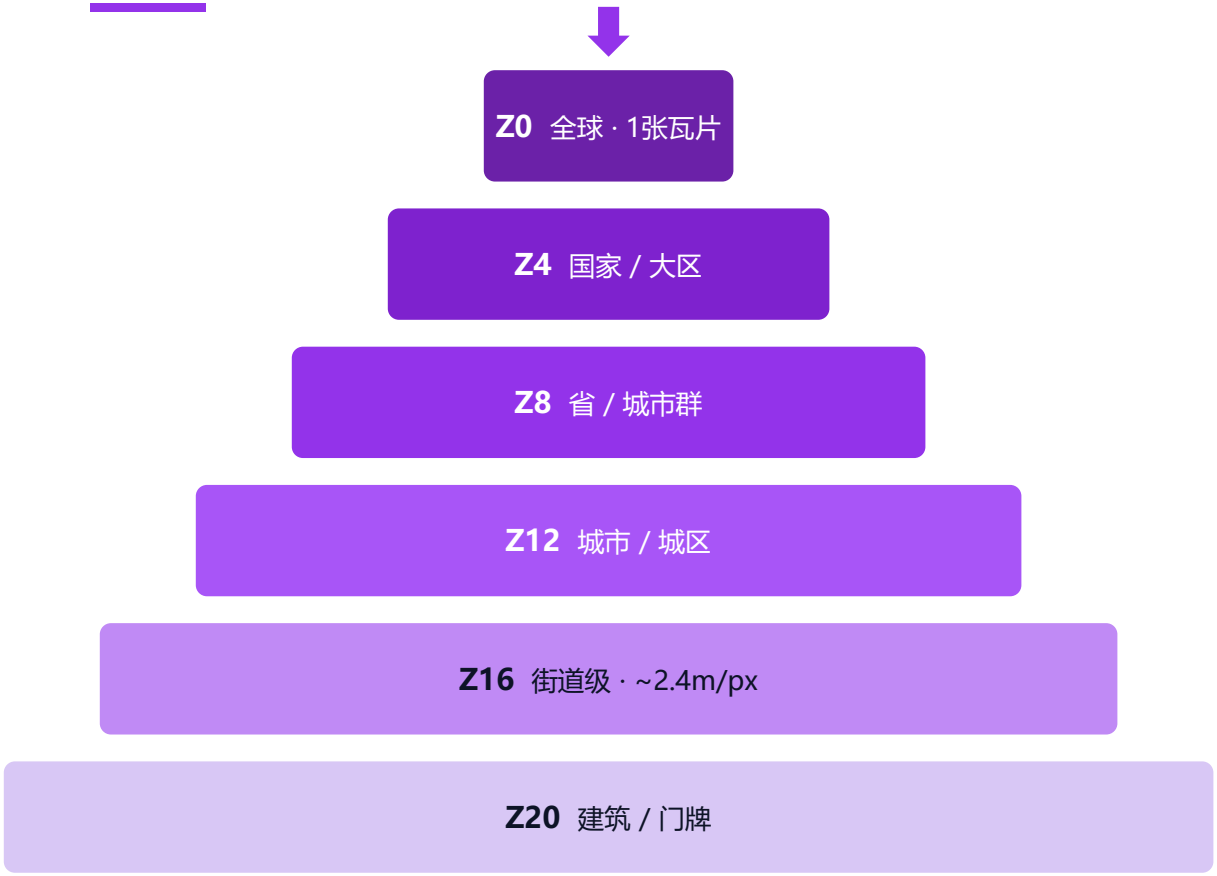
瓦片金字塔结构

• 全球网格化切分，从zoom 0到zoom 22+• zoom 0：1张瓦片覆盖全球（256x256）• 每升一级，瓦片数量翻4倍• zoom n：2^n × 2^n 张瓦片• 瓦片坐标：(x, y, z) 三元组• Web墨卡托投影下的正方形瓦片• 常见最大级别：道路16级，影像20级 zoom 10：城市级（约150m/像素） zoom 16：街道级（约2.4m/像素）

LOD调度策略

• LOD = Level of Detail，细节层次• 根据缩放级别选择合适的瓦片层级• 缩放过渡时：高层级渐入，低层级渐出• 缩放时先放大低层级瓦片，再加载高层级• 视口内瓦片优先级 > 视口外瓦片• 预取策略：移动方向预判，提前加载• 降级策略：网络差时使用低层级瓦片 常见LOD层级：全球→国家→城市→街道 每个层级有对应的要素简化级别

瓦片金字塔模型与LOD调度



每升一级瓦片数×4：zoom n = $2^n \times 2^n$ 张瓦片

瓦片坐标 (x, y, z) · Web墨卡托正方形瓦片

LOD 调度策略

视口优先级	视口内瓦片 > 视口外；预判移动方向提前加载（预取）
缩放过渡	先放大低层级瓦片显示，再渐入高层级——避免白屏
降级策略	网络差时用低层级瓦片兜底，保证可用性
要素简化	每个层级有对应简化级别：道路16级、影像20级封顶

层级速查：zoom 10 城市级（约150m/像素）· zoom 16 街道级（约2.4m/像素）

全球→国家→城市→街道，每级有对应要素简化级别

Web墨卡托投影与坐标变换

Web墨卡托投影

• EPSG:3857 / EPSG:900913 标准• 圆柱投影，将球面投影到圆柱面再展开• 正轴等角圆柱投影，保持角度不变• 纬度方向有面积变形，高纬度被拉伸• 纬度范围限制在±85.0511287798°• 投影后为正方形，方便瓦片切分• x轴：经度线性映射 0~2πR• y轴：ln(tan(φ/2 + π/4)) 非线性映射

公式：x = R * (λ - λ0) 公式：y = R * ln(tan(π/4 + φ/2))

坐标变换链

• WGS84经纬度 → Web墨卡托米制坐标• 墨卡托坐标 → 世界像素坐标（zoom相关）• 世界像素坐标 → 瓦片坐标(x,y,z)• 瓦片坐标 → 瓦片内像素坐标• 瓦片内坐标 → 几何顶点坐标• MVP矩阵变换 → 屏幕像素坐标• Model矩阵：瓦片偏移+要素内偏移• View矩阵：相机位置/旋转/倾斜• Projection矩阵：透视或正交投影

完整链路：经纬度 → 世界 → 视图 → 裁剪 → 屏幕

坐标系统

Web墨卡托投影——把地球摊平成方形瓦片世界

投影公式（球面近似）

$x = R \cdot \lambda$ （经度线性映射）

$y = R \cdot \ln(\tan(\pi/4 + \varphi/2))$

纬度截断：±85.0511°（让世界成为正方形）

为什么渲染引擎都选它

等角（conformal）：方向不变形，导航转向判断正确

线性经度：瓦片网格可直接按 z/x/y 切分

代价：高纬面积放大（格陵兰≈非洲大小），测距必须换算

等角

方向正确，适合导航

方形世界

z/x/y 瓦片编号成立

分辨率

156543·cos(lat)/2^z 米每像素

工程要点：WGS84经纬度 → 墨卡托米 → 瓦片号的转换函数必须放在坐标系模块唯一入口，禁止散落在渲染代码里

瓦片坐标系统与视口瓦片计算

瓦片坐标计算

- 给定经纬度(lng, lat)和缩放级别z
- x = floor((lng + 180) / 360 * 2^z)
- y = floor((1 - ln(tan(lat) + 1/cos(lat))/π) / 2 * 2^z)
- 瓦片大小：256×256像素（标准）
- 瓦片内坐标：0~255 或 0~4095（MVT）
- X轴从西向东递增，Y轴从北向南递增
- 瓦片边界：左闭右开 [x, x+1)

例：zoom=1, x=0, y=0 → 左上四分之一地球 例：北京 (116.4°, 39.9°) zoom=10 → (846, 396, 10)

视口瓦片计算

- 根据屏幕范围计算需要加载的瓦片
- 步骤1：屏幕四角 → 经纬度坐标
- 步骤2：经纬度 → 瓦片坐标 (x,y,z)
- 步骤3：计算x范围和y范围
- 步骤4：生成瓦片请求列表
- 考虑旋转和倾斜：需要计算包络矩形
- 旋转时：计算旋转后的外接矩形
- 倾斜时：考虑梯形视口的扩大

边缘扩展：多加载一圈瓦片防闪烁 优先级：中心瓦片优先，边缘瓦片延后

Mapbox Style规范与样式系统设计

样式规范核心概念

- Mapbox Style Specification v8 标准
- JSON格式描述完整地图样式
- sources：数据源定义（矢量/栅格/GeoJSON）
- layers：渲染图层列表，按顺序渲染
- sprite：sprite图（图标合集）
- glyphs：字形（SDF字体）
- filters：图层过滤条件
- paint：绘制属性（颜色、线宽、透明度等）
- layout：布局属性（显隐、符号放置等）

图层类型体系

- background：背景图层
- fill：填充图层（面要素）
- line：线图层（道路、边界等）
- symbol：符号图层（文字+图标）
- circle：圆点图层（POI等）
- fill-extrusion：3D挤出图层
- raster：栅格图层（影像、地形）
- heatmap：热力图图层
- hillshade：山体阴影图层

表达式求值引擎——数据驱动样式的核心

表达式语法

- JSON数组形式：["op", arg1, arg2, ...]
- 支持嵌套表达式，形成表达式树
- 类型系统：number/string/boolean/color
- 字面量：["literal", value]
- 属性获取：["get", "property_name"]
- 数学运算：["+", a, b] ["*", a, b] 等
- 比较运算：[">", a, b] ["==", a, b] 等
- 逻辑运算：["all", a, b] ["any", a, b]
- 条件分支：["case", cond1, val1, cond2, val2, default]

插值：["interpolate", ["linear"], ["zoom"], 10, 1, 16, 5]

求值引擎实现

- 递归下降求值：遍历表达式AST
- 编译优化：表达式编译为函数/JIT
- 数据驱动：属性值随要素属性变化•
缩放驱动：属性值随zoom级别变化
- 复合驱动：同时依赖数据和缩放
- 缓存机制：相同输入缓存输出结果
- 类型检查：运行时类型推断与校验
- 性能优化：热点表达式预编译

典型场景：道路等级→线宽、行政级别→字号

进阶：函数式样式编程，无限灵活性

运行时样式切换——日/夜/导航模式

三种典型样式模式

- 日间模式（Day Mode） 浅色背景，高对比度，适合户外
道路分级清晰，水系绿地丰富
- 夜间模式（Night Mode） 深色背景，低亮度，减少眩光
暖色道路，降低整体亮度
- 导航模式（Navigation Mode） 突出当前道路和转向箭头
弱化背景信息，强化引导元素 车头朝上，3D倾斜视角

样式切换实现机制

- 样式表切换：加载新的style JSON
- 增量更新：只更新变化的图层属性
- 过渡动画：颜色/透明度渐变过渡
- 过渡时间：通常300-500ms
- 双缓冲：新样式渲染完成后再切换
- 图层级切换：可单独切换某类图层
- 数据复用：切换样式不重新加载瓦片
- GPU资源复用：纹理/缓冲尽量复用

工程实践：预加载常用样式避免卡顿

车机场景：进隧道自动切换夜间模式

样式系统性能优化策略

渲染性能优化

- 图层排序：同类图层合并，减少状态切换
- 图层过滤：先过滤后计算，减少处理量
- 样式预计算：缩放相关属性预计算表
- 表达式编译：热点表达式编译为字节码
- 数据驱动样式：减少JS调用，GPU侧计算
- 属性批量计算：批量处理要素属性
- GPU着色器中计算：简单逻辑放入shader

颜色插值：在片元着色器中完成

线宽缩放：在顶点着色器中完成

内存与加载优化

- 样式按需加载：只加载当前需要的图层
- 雪碧图合并：图标合并减少纹理数量
- 字体子集化：按需加载文字字形
- 样式缓存：解析后的样式AST缓存
- 样式懒加载：图层可见时才解析样式
- 样式版本管理：增量更新避免全量重载
- sprite图纹理压缩：减少GPU内存

工程实践：样式lint工具检查性能问题

避免过多图层、过复杂表达式

地图文字标注的三种基本类型

点标注（Point Annotation）

- 适用于POI、地名、站点等点状要素
- 标注位置：点要素的坐标位置
- 偏移设置：文字相对于点的偏移量
- 图标+文字组合：图标在上，文字在下
- 对齐方式：居中、左对齐、右对齐
- 避让规则：优先避让高优先级要素
- 堆叠策略：同类标注堆叠显示数量

常见问题：密集区域标注重叠严重 解决思路：优先级排序 + 碰撞检测

线标注 & 面标注

- 线标注（Line Annotation） 沿道路/河流等线状要素排列文字 文字跟随路径方向旋转 长道路上重复放置多个标注 弯曲度限制：转角太急不放置
- 面标注（Area Annotation） 行政区、湖泊等面状要素标注 放置在面的骨架线或重心位置 文字大小随面面积自适应 跨瓦片标注需特殊处理避免截断

标注碰撞检测算法与优先级策略

碰撞检测算法

- 包围盒碰撞（AABB）：最常用，矩形检测
- 圆形碰撞：适用于图标类标注
- 多边形碰撞：精确但计算量大
- 贪心算法：按优先级逐个放置检测
- 空间网格加速：将屏幕网格化，O(1)查询
- 四叉树加速：空间索引加速碰撞查询
- 检测步骤：计算标注框 → 查询空间索引 → 精细检测

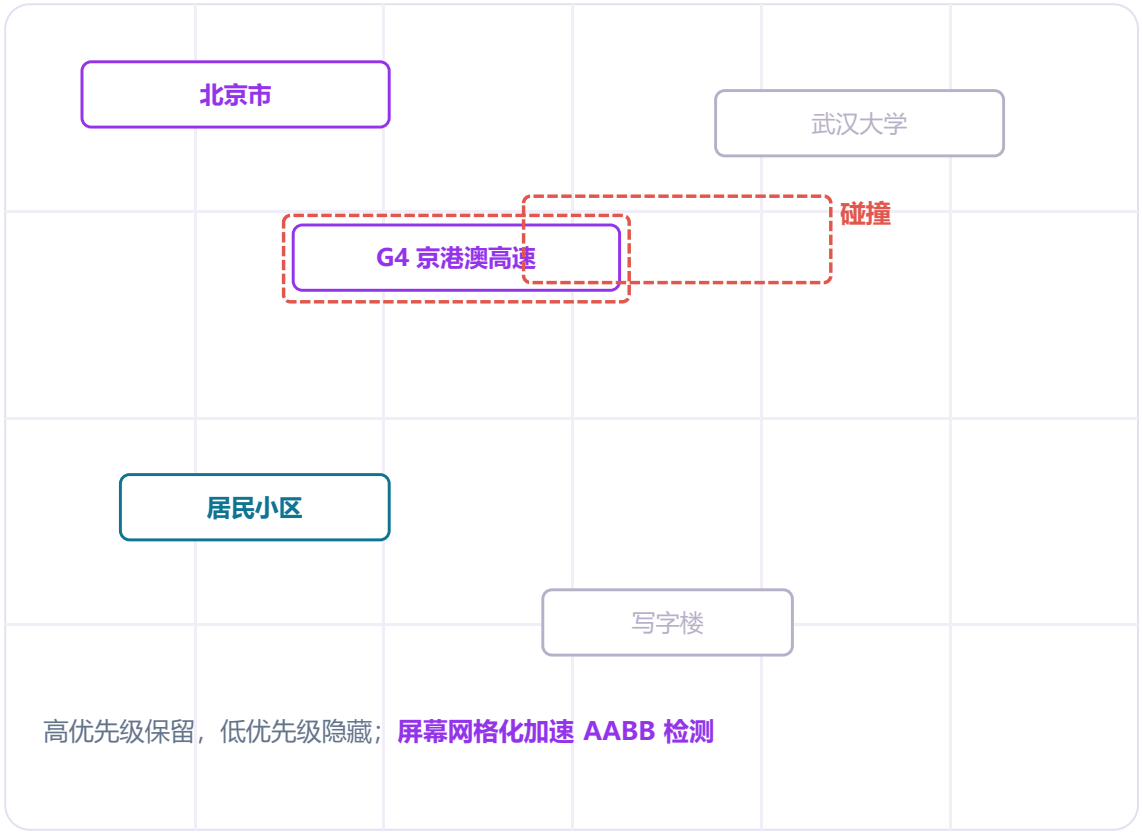
时间复杂度：贪心$O(n^2)$ → 空间索引$O(n \log n)$ 典型性能：单帧处理数千个标注

标注优先级策略

- 优先级决定标注显示顺序• 要素类型优先级：行政区>道路>POI
- 行政级别：国家级>省级>市级>区县
- 道路等级：高速>国道>省道>县道
- 人口/重要性：大城市优先显示
- 用户设置：收藏/常用地点优先
- 碰撞时：高优先级保留，低优先级隐藏
- 同优先级：按位置或重要性评分决定

渐进显示：缩放时逐步增加标注密度 体验优化：标注尽量不跳动、不闪烁

标注碰撞检测与优先级策略



优先级决定显示顺序

行政区标注	国家级 > 省级 > 市级 > 区县
道路标注	高速 > 国道 > 省道 > 县道
POI 标注	大城市/重要度优先 · 收藏优先

检测流程（贪心 + 空间索引）

计算标注包围盒 → 查询空间网格索引 → AABB 精细检测 → 按优先级放置/隐藏

复杂度：贪心 $O(n^2)$ → 空间索引 $O(n \cdot \log n)$，单帧可处理数千标注

高级标注放置算法

点标注放置优化

- 8方向候选位置：上/下/左/右/四对角
- 评分函数：位置偏好 + 遮挡惩罚
- 优先选择无遮挡的最佳位置
- 文字换行：长文本自动换行显示
- 文字缩写：空间不足时使用简称
- 聚合标注：同类POI聚合成数字标注
- 动态避让：随地图移动实时重排

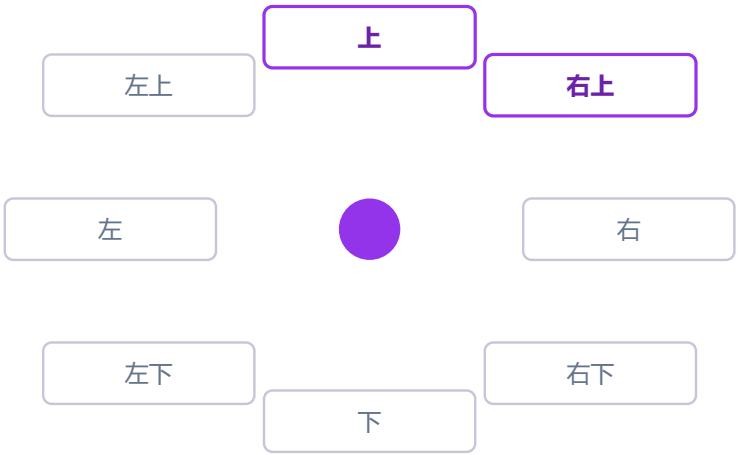
工程难点：标注稳定性（避免抖动） 解决：位置锁定 + 渐入渐出

线标注沿路径放置

- 路径采样：沿线等距采样候选位置
- 文字长度计算：计算文字占据的路径长度
- 弯曲度检测：计算路径曲率，太弯跳过
- 重复放置：长路径上等间距重复标注
- 朝向选择：文字方向与路径方向一致
- 避让检测：与其他标注的碰撞检测
- 跨瓦片处理：长路径跨瓦片需协调 算法：沿路径滑动窗口找最佳放置段 优化：预计算路径的可放置段落

高级标注放置算法——点标注与线标注

点标注：8 方向候选位置 + 评分函数



评分 = 位置偏好 + 遮挡惩罚；无遮挡则选最佳位置，空间不足自动换行/缩写；同类POI聚合成数字标注

线标注：沿路径放置与重复标注



▲ 沿线路径：等距采样 → 滑动窗口找可放置段 → 弯曲度太弯跳过

朝向与路径一致 · 长路径等间距重复 · 跨瓦片需协调；工程难点：标注稳定性——位置锁定 + 渐入渐出防抖动

渐进显示：随缩放逐步增加标注密度 • 体验目标：标注尽量不跳动、不闪烁

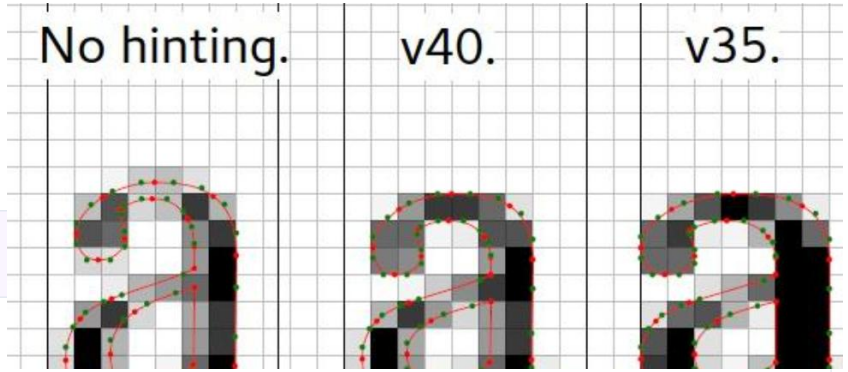
有向距离场文字渲染技术

SDF原理

- SDF = Signed Distance Field，有向距离场
- 每个像素存储到最近文字轮廓的距离
- 正值在轮廓内，负值在轮廓外
- 传统位图文字：缩放有锯齿，需多套字号
- SDF文字：单张纹理，任意缩放清晰
- MSDF = Multi-channel SDF，多通道距离场
- MSDF用RGB三通道存储不同方向的距离
- MSDF解决了SDF在尖角处的模糊问题

字体生成：fontbm / msdf-atlas-gen 工具 存储：每个字形存储在雪碧图中

SDF渲染实现

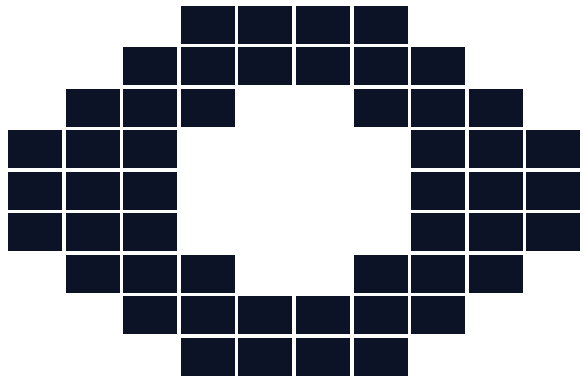


- 片元着色器中采样SDF纹理
- 阈值判断：alpha = smoothstep(edge-0.5, edge+0.5, dist)
- 边缘抗锯齿：smoothstep实现平滑边缘
- 描边效果：第二个阈值实现文字描边
- 阴影效果：偏移采样实现文字阴影• 光晕效果：距离渐变实现发光效果
- 性能：单pass渲染，GPU友好

优势：一套字体支持任意缩放、描边、阴影 局限：小字质量略逊于位图，生成成本高

有向距离场（SDF）文字渲染原理

传统位图文字

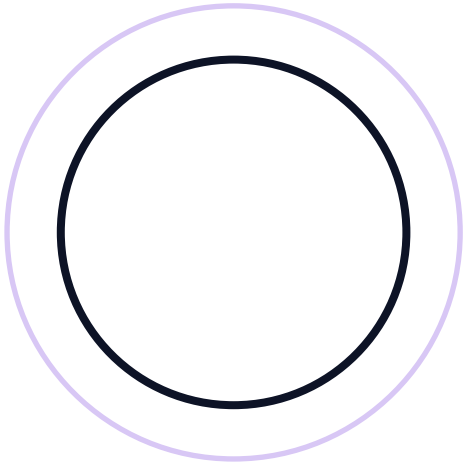


放大 → 锯齿明显（马赛克边缘）

每种字号都要生成一套位图纹理

缩放过渡时需要多套字号 + 模糊过渡

SDF 文字



正值：轮廓内

0：轮廓边界

负值：轮廓外

每像素存

到轮廓的距离

```
alpha = smoothstep(edge−w, edge+w, dist)
```

单张纹理任意缩放都清晰 · smoothstep 实现边缘抗锯齿

第二阈值→描边 · 偏移采样→阴影 · 距离渐变→光晕；MSDF 用RGB三通道解决尖角模糊

字体生成：fontbm / msdf-atlas-gen → 字形雪碧图 • 单pass渲染 GPU友好 • 局限：小字略逊位图、生成成本高

地图交互模式与手势识别

核心交互模式

- 平移（Pan）：单指拖动地图
- 缩放（Zoom）：双指捏合或双击
- 旋转（Rotate）：双指旋转手势
- 倾斜（Tilt）：双指上下滑动调整倾角
- 点击（Tap）：单击选择要素
- 双击（Double Tap）：双击放大
- 长按（Long Press）：长按选择/添加标注
- 惯性滚动：抬手后继续滑动减速

地图自由度：2D（平移+缩放） / 2.5D（+旋转+倾斜） / 3D

手势识别系统

- 触摸事件→手势识别器→地图操作
- 识别器状态机：Possible→Began→Changed→Ended
- 多手势协同：平移+缩放+旋转同时进行
- 手势优先级：处理手势冲突
- 触摸点管理：单指/双指/多指触点跟踪
- 速度计算：滑动速度决定惯性初速度
- 边界检测：地图边界、缩放范围限制
- 弹性效果：超出边界后回弹动画

工程实践：iOS UIGestureRecognizer / Android MotionEvent Web端：Pointer Events / Touch Events

惯性滚动与捏合缩放实现原理

惯性滚动实现

- 抬手时记录瞬时速度 v0
- 减速模型：v(t) = v0 * e^(-kt) 或线性衰减
- 位移积分：s(t) = ∫v(t)dt
- 指数衰减：自然减速，体验更顺滑
- 线性衰减：计算简单，便于预测
- 摩擦系数：控制减速快慢（0.92~0.98）
- 停止条件：速度低于阈值或达到边界
- 边界回弹：超出边界后弹性返回

动画帧率：与渲染帧率同步（60fps） 性能：纯CPU计算，每帧更新相机位置

捏合缩放实现

- 双指距离变化计算缩放比例
- 缩放中心：双指中点位置
- 缩放公式：newScale = oldScale * (dist2 / dist1)
- 锚点保持：缩放中心的地图位置不变
- 缩放范围限制：minZoom ~ maxZoom
- 缩放阻尼：接近边界时缩放变慢
- 平滑动画：缩放结束后的弹性动画
- 双击缩放：快速放大一级，有动画过渡

双指旋转：计算两指角度差，更新地图旋转角 组合手势：平移+缩放+旋转同时进行

性能优化

地图渲染性能优化四大策略

视锥体剔除

只渲染视口可见的瓦片和要素，大幅减少绘制量，是最有效的优化手段

批量渲染

合并同类要素的Draw Call，减少GPU状态切换，提升渲染吞吐量

多级缓存

瓦片数据、几何缓冲、纹理资源多级缓存，LRU策略管理内存

渐进式渲染 · 先粗后细、先快后精，优先保证可交互性，逐步提升画质

减少Draw Call的核心技术

Draw Call为什么昂贵

- Draw Call = 一次GPU绘制调用
- CPU开销：状态设置、驱动验证、命令提交
- 每次Draw Call需要设置：着色器、纹理、顶点缓冲....
- Draw Call太多会导致CPU成为瓶颈
- 移动端GPU对Draw Call更敏感
- 经验法则：移动端每帧<300个Draw Call
- 批量渲染的目标：合并同类绘制调用

GPU是并行处理器，批量越大效率越高 性能瓶颈常出在CPU端而非GPU端

Draw Call合并技术

- 顶点缓冲合并：同图层要素合并到一个VBO
- 纹理图集（Atlas）：多图标合并到一张纹理
- 实例化渲染（Instancing）：相同几何体多次绘制
- Uniform Buffer Object：批量设置Uniform变量
- 图层排序：同材质图层连续渲染减少状态切换
- 状态缓存：相同状态不重复设置
- 自动批处理：引擎自动合并可合并的绘制项

代价：合并增加内存，需要权衡 策略：静态数据合并，动态数据分开

LRU缓存策略与多级缓存体系

LRU缓存原理

- LRU = Least Recently Used，最近最少使用
- 缓存满时淘汰最久未使用的条目
- 数据结构：HashMap + 双向链表
- HashMap：O(1)查找缓存条目
- 双向链表：维护使用顺序，O(1)移动
- 访问时：移到链表头部（最新）
- 淘汰时：删除链表尾部（最旧）
- 容量控制：按数量或内存大小限制

变体：LRU-K、2Q、ARC 等改进算法 瓦片缓存是LRU最经典的应用场景

多级缓存体系

- L1：GPU内存缓存——纹理、顶点缓冲
- L2：内存缓存——解析后的瓦片数据
- L3：本地磁盘缓存——原始瓦片文件
- L4：网络请求——远程服务器下载
- 缓存命中率是关键指标
- 预取策略：预判用户移动方向提前加载
- 瓦片优先级：当前视口 > 附近区域
- 内存警告时降级：释放低级别缓存

典型命中率：内存70%+，磁盘90%+ 工程实践：分层独立LRU，各自容量控制

多级瓦片缓存体系——让90%的请求不碰网络

L1 内存缓存
瓦片位图/解码后纹理 · 容量128MB · 命中<1ms
当前视口正在显示的瓦片常驻

L2 磁盘缓存
序列化MVT/位图 · 容量500MB~2GB · 命中5~20ms
跨会话保留，冷启动也能秒开

L3 网络请求
CDN拉取 · 200~800ms · 兜底
仅15%的请求走到这里

L3命中后逐级回写：网络→磁盘→内存（下次同瓦片直接L1命中）

淘汰策略	内存/磁盘都用LRU：最近最少使用的瓦片先淘汰 · 预取视野周边瓦片（外扩1圈）可显著提升命中率
缓存键设计	z/x/y + 样式版本号 + 数据源ID——样式切换不串缓存 · 离线地图=预填充的L2缓存
典型命中率	内存35% + 磁盘50% + 网络15% · 命中率每提升10%，平移/缩放的掉帧率约下降一半

与作业的衔接：任务3的缓存分=LRU实现正确性（容量/淘汰/回写）+ 命中率报告

渐进式渲染策略——先快后精

渐进渲染思路

- 感知性能 > 绝对性能
- 用户更在意是否能立即看到内容
- 策略：先显示低质量内容，逐步优化
- 瓦片级渐进：低层瓦片先显示，高层级逐步加载
- 要素级渐进：先渲染简单要素，再补充复杂要素
- 质量级渐进：低分辨率→高分辨率
- 首屏时间是关键指标

目标：首屏<1秒，完整加载<3秒 原则：用户永远看到的是'可用'的地图

具体实现手段

- 低层级瓦片放大显示：高层级未加载时放大低层级
- 骨架渲染：先渲染色块，再叠加细节
- 分图层加载：道路→POI→文字，分阶段显示
- 降质渲染：低端设备降低渲染质量
- 帧率自适应：动态调整渲染负载
- 懒加载：非关键区域延后加载
- 占位符：加载中显示简化版本

性能监控：实时统计帧率和加载时间 动态调优：根据设备性能调整策略

低空无人机视角地图渲染

第一人称视角渲染

- FPV = First Person View，第一人称视角
- 相机位置：无人机机身位置• 相机姿态：跟随无人机姿态变化
- 大视角广角：模拟无人机摄像头视野
- 地平线渲染：天空与地面的交界
- 高度感知：通过视角倾斜体现飞行高度
- 速度感：通过运动模糊/视差体现速度

渲染挑战：地形起伏、障碍物检测 应用场景：无人机巡检、测绘、航拍

地形跟随与障碍物可视化

- 地形数据：DEM数字高程模型
- 地形采样：沿飞行路径采样地形高度
- 地形贴合：相机高度随地形起伏调整
- 障碍物检测：识别高于安全高度的地物
- 障碍物高亮：红色警示框标注危险物
- 高度线渲染：显示当前高度与地面距离
- 航迹显示：已飞行路径和规划路径

安全围栏：可视化禁飞区和限高区 实时性要求：延迟<100ms，否则影响操控

第一人称视角渲染技术要点

相机模型

- 透视投影：模拟人眼/相机透视效果
- FOV（视场角）：通常60°~90°广角
- 相机姿态：pitch（俯仰）+ yaw（偏航）+ roll（横滚）
- 姿态更新：IMU数据驱动相机姿态
- 平滑滤波：姿态数据低通滤波防抖动
- 近裁剪面：避免相机穿入地面
- 远裁剪面：控制可见距离，优化性能

雾效：远处逐渐淡出，增加空间感

景深：模拟真实相机的焦点效果

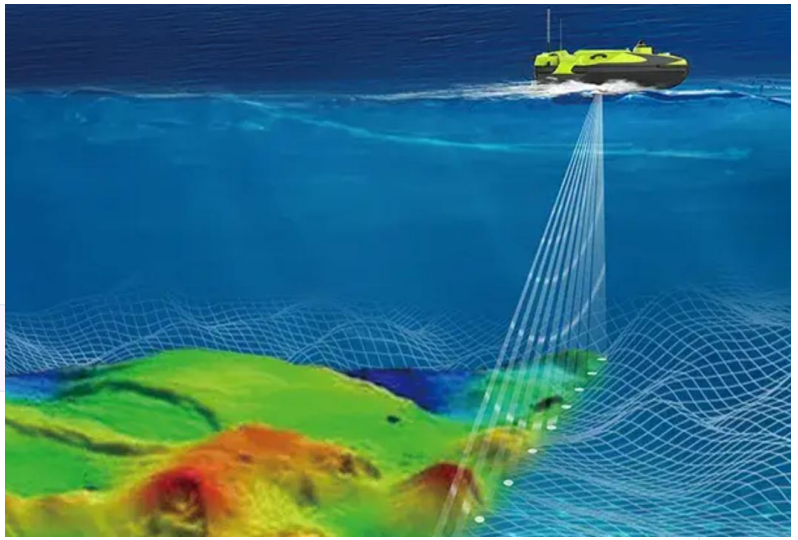
渲染管线调整

- 地形渲染：高精度地形网格 + 纹理采样
- 地表纹理：卫星影像 + 地形纹理混合
- 地物渲染：3D建筑、树木、电力线等
- 气象效果：云、雾、雨、雪粒子系统
- 光照计算：太阳位置动态光照
- 阴影渲染：地形阴影增强立体感
- HUD叠加：高度、速度、姿态等仪表

性能挑战：地形数据量大，需LOD调度

优化：GeoMipMapping、Chunked LOD

水下AUV航行可视化



水柱可视化

- 水体渲染：半透明水体积渲染
- 深度着色：深度越大颜色越深
- 水体积雾：水下能见度有限，远处渐隐
- 光线散射：丁达尔效应（光柱效果）
- 水面折射：水面波动的折射效果
- 海底地形：水下DEM地形渲染
- 海流可视化：流线/粒子表示洋流方向

颜色体系：深蓝/青绿为主色调

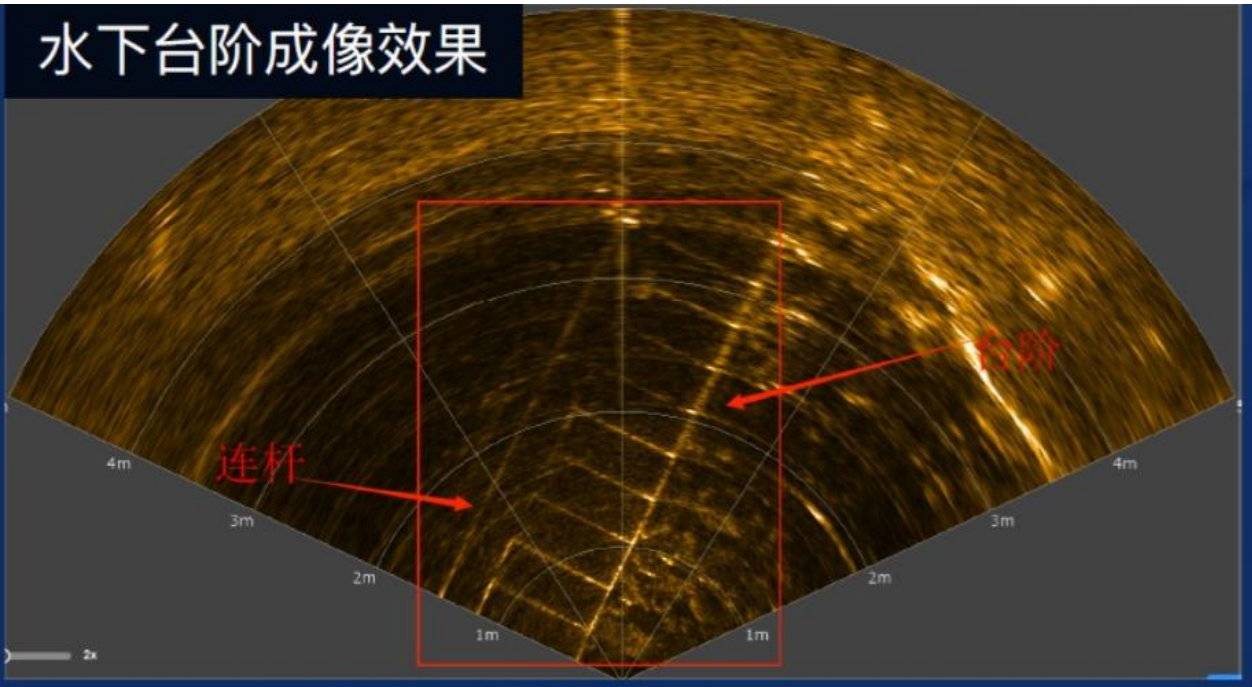
挑战：水下光学特性复杂，逼真渲染难

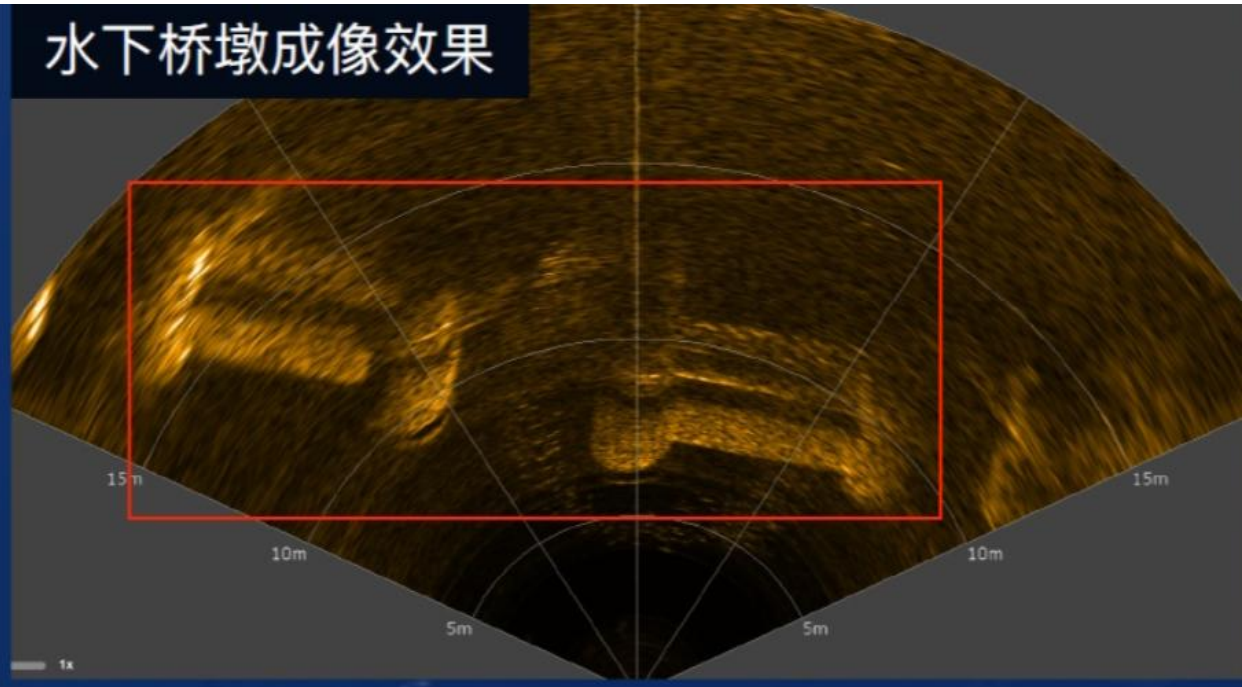
AUV状态显示

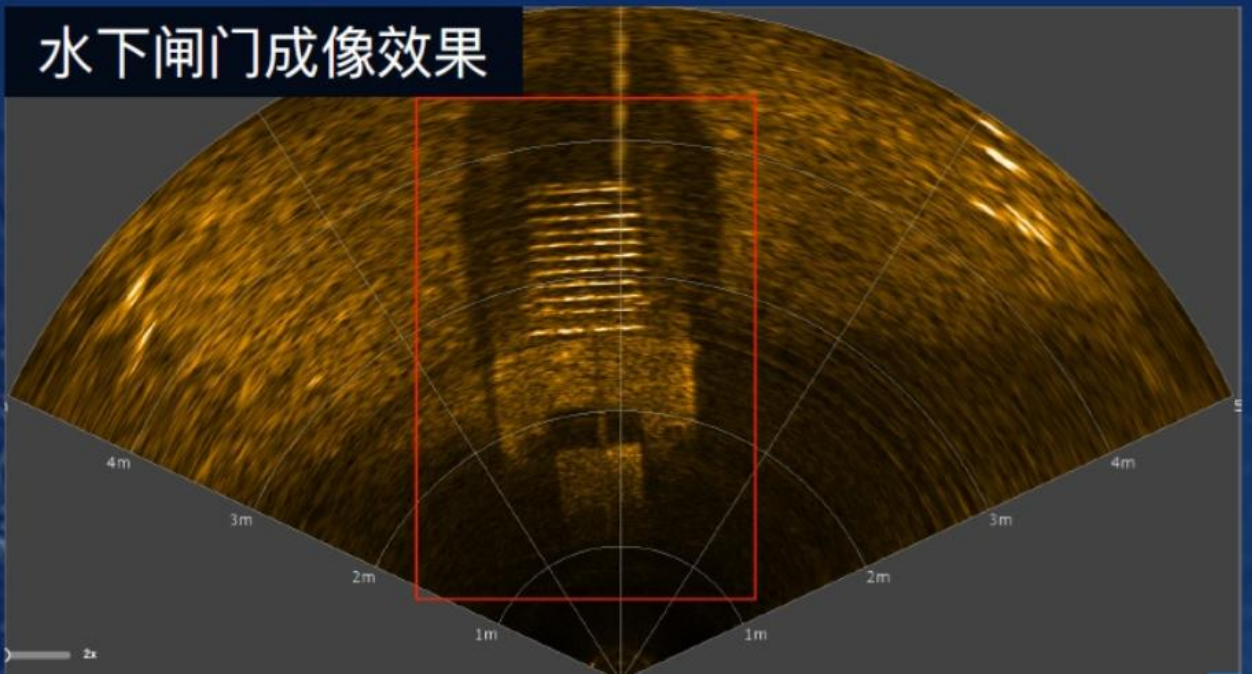
- AUV三维模型：显示机器人姿态
- 航迹线：历史航行路径轨迹
- 深度指示：当前深度与深度刻度
- 姿态仪表：俯仰角、偏航角、横滚角
- 速度向量：速度大小和方向指示
- 电池电量：续航状态可视化
- 通信状态：与母船的连接状态

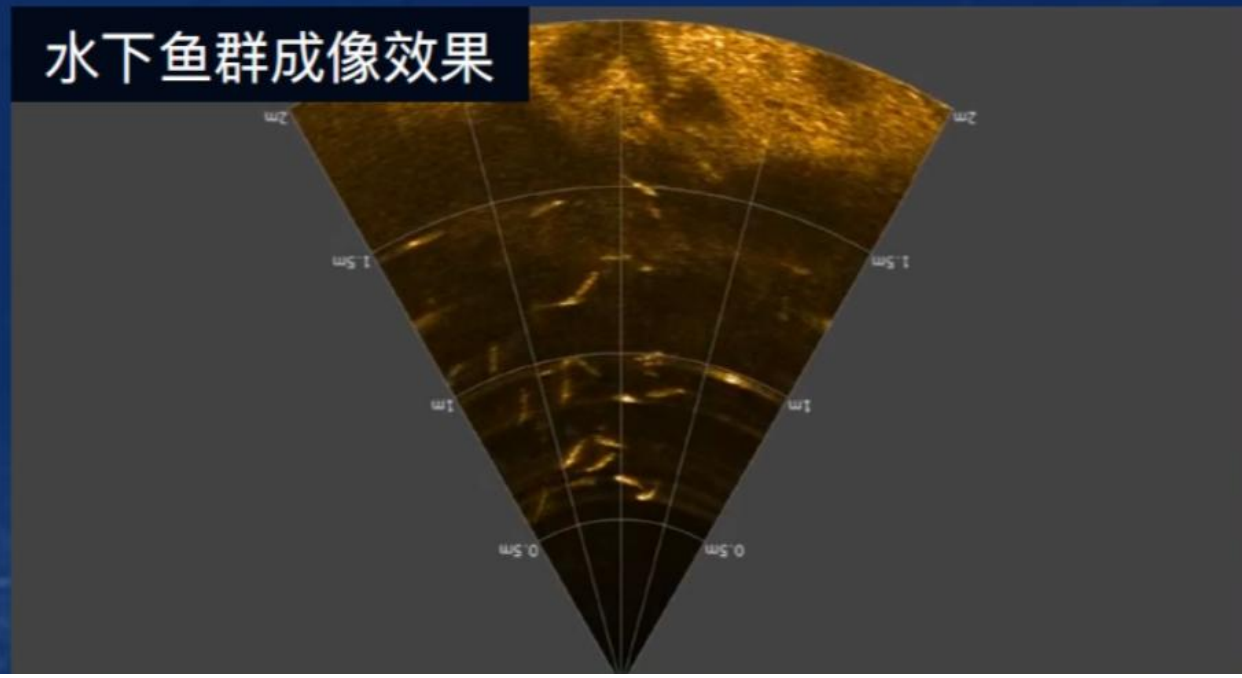
任务状态：当前任务阶段和进度

预警信息：故障、超深、失联等告警


水下台阶成像效果
连杆
台阶
4m
3m
2m
1m


水下桥墩成像效果
15m
10m
5m


水下闸门成像效果
4m
3m
2m
1m


水下鱼群成像效果

行业案例

Mapbox GL —— 现代矢量瓦片渲染引擎标杆

Mapbox GL架构特点

- Mapbox GL JS：Web端矢量瓦片渲染开创者
- Mapbox GL Native：C++原生渲染引擎
- 跨平台：Web/iOS/Android/macOS/Linux
- 完全基于矢量瓦片和样式规范
- GPU加速渲染：WebGL/Metal/Vulkan
- 样式驱动：JSON样式描述一切
- 插件生态：丰富的第三方扩展

核心贡献：定义了MVT和Style Spec标准

影响：几乎所有现代地图引擎都受其影响

技术亮点

- TileJSON：瓦片元数据标准格式
- GeoJSON源：支持实时数据渲染
- 表达式系统：强大的数据驱动样式
- Terrain RGB：地形高程编码到RGB
- 3D建筑：fill-extrusion挤出建模
- Sky API：天空和大气渲染
- Projection API：支持多种投影切换

性能优化：Worker线程解析瓦片

离线能力：移动端离线包支持

国内厂商

高德地图 vs 百度地图渲染引擎对比

高德地图

Amap / AutoNavi

· 渲染引擎 · 自研AMap引擎，V7版本引入矢量瓦片
· 技术路线 · 侧重车机导航体验，3D渲染能力强
· 性能特点 · 车机端优化深入，低性能设备适配好
· 数据特色 · 道路数据精细，导航体验出色
· 3D能力 · 3D视界、城市级3D建筑

百度地图

Baidu Maps

· 渲染引擎 · 自研渲染引擎，逐步从栅格转向矢量
· 技术路线 · 侧重POI和AI能力，AR导航特色
· 性能特点 · 移动端体验流畅，动画效果丰富
· 数据特色 · POI数据丰富，室内地图领先
· 3D能力 · 3D街景、全景地图

共同特点

国内厂商技术特色

· 渲染引擎 · 都经历了从栅格到矢量的演进
· 技术路线 · 都重视3D城市和实景导航
· 性能特点 · 都针对国内设备环境深度优化
· 数据特色 · 数据质量是渲染效果的基础
· 3D能力 · 都在大力投入3D和实景化

💡 **国内厂商在数据质量、本地化体验、车机适配方面积累深厚，但引擎底层技术（如样式规范、跨平台架构）仍受Mapbox GL影响较深。**

Google Maps渲染技术——行业天花板

技术架构

- Google Maps JS API / Native SDK
- 矢量瓦片为主，栅格瓦片为辅
- WebGL渲染，全3D架构
- Photorealistic 3D：照片级真实感3D城市
- Immersive View：沉浸式街景+3D融合
- Live View：AR实景导航
- 全球覆盖，数据量极其庞大

渲染质量：业界标杆，细节极其丰富

流畅度：低端设备上也能流畅运行

核心技术亮点

- 矢量压缩：自研高效矢量压缩格式
- 渐进加载：多级分辨率渐进式加载
- 智能标注：AI优化的标注布局算法
- 实时更新：地图数据近乎实时更新
- 3D重建：大规模城市3D自动重建
- 街景融合：街景图像与3D模型融合
- 机器学习：AI辅助地图生成与优化

性能秘诀：高度优化的C++引擎 + GPU加速

研发投入：数千人团队，数十年积累

地图渲染引擎的跨平台方案

跨平台技术路线

• 方案一：C++核心 + 各端原生封装 Mapbox GL Native 模式 核心渲染逻辑C++实现，各平台薄封装 性能好，开发成本较高

• 方案二：Web技术 + 容器 基于WebGL，Electron/WebView容器 开发效率高，性能略弱

• 方案三：Flutter/React Native 跨平台UI框架 + 原生渲染插件 折中方案，适合快速迭代

渲染后端抽象

• 抽象图形层（GAL = Graphics Abstraction Layer）
• 统一接口，多后端实现
• WebGL：Web端、移动端WebView
• OpenGL ES：Android、iOS旧版
• Metal：iOS/macOS，性能最优
• Vulkan：Android新设备、Linux
• D3D11：Windows桌面

着色器跨平台：GLSL → 各平台着色语言 工具：SPIR-V-Cross、MoltenVK

开源工具

主流开源地图渲染引擎对比



MapLibre GL

Mapbox GL社区分支

定位	生产级矢量瓦片渲染引擎
渲染技术	WebGL GPU加速渲染
矢量瓦片	原生支持，MVT标准
3D能力	3D建筑、地形、倾斜视角
性能	优秀，适合大数据量
学习曲线	中等，样式系统复杂



OpenLayers

Web端GIS渲染库

定位	全功能GIS Web渲染库
渲染技术	Canvas + WebGL混合
矢量瓦片	支持，需插件
3D能力	有限3D支持
性能	良好，GIS场景优化
学习曲线	较陡，GIS概念多



Leaflet

轻量级Web地图库

定位	轻量易用的Web地图库
渲染技术	SVG/Canvas，以DOM为主
矢量瓦片	支持，需插件
3D能力	基本无3D能力
性能	一般，适合轻量场景
学习曲线	平缓，API简单直观

Mapnik（补充）：服务端地图渲染引擎，C++实现，用于生成瓦片图片，是OSM等项目的后端渲染引擎。

MapLibre GL深入——架构与源码学习

核心架构模块

- Style：样式解析与管理
- Source：数据源管理（矢量/栅格/GeoJSON）
- Tile：瓦片加载、解析、缓存
- Layer：图层渲染逻辑
- Painter：WebGL渲染实现
- Transform：相机/视图变换
- Render：渲染调度与帧循环
- Interaction：交互与手势处理

Worker：后台线程解析瓦片数据

代码量：约10万行TypeScript/JS

学习路径建议

- 第一步：理解样式规范（Style Spec）
- 第二步：掌握MVT瓦片格式
- 第三步：阅读Style和Source模块
- 第四步：理解Tile生命周期
- 第五步：深入Painter渲染实现
- 第六步：研究性能优化策略
- 实践：修改样式、添加自定义图层
- 进阶：实现自定义渲染插件

推荐：从MapLibre GL JS入手，代码可读性好 Native版本适合学习高性能C++渲染架构

Ear Clipping耳切法——多边形三角化

为什么需要三角剖分

• GPU只能渲染三角形• 面要素（多边形）必须拆分为三角形• 三角剖分质量决定渲染效率• 三角剖分速度影响加载性能• 简单多边形：无自交的多边形• 带洞多边形：需要特殊处理• 地图中的面要素：行政区、建筑、水系等 目标：将n边形分解为n-2个三角形 约束：三角形不重叠、不遗漏、共顶点

耳切法（Ear Clipping）

• 算法思想：不断切除多边形的'耳朵'• 耳朵：一个顶点及其相邻两顶点构成的三角形• 耳朵判定：三角形内部不含其他顶点• 算法步骤： 1. 建立顶点链表（双向循环链表） 2. 找出所有耳朵顶点 3. 切下一个耳朵，输出三角形 4. 更新相邻顶点的耳朵状态 5. 重复直到只剩3个顶点 时间复杂度：简单实现$O(n^2)$，优化O(n log n)

Douglas-Peucker算法——道格拉斯-普克

为什么需要几何简化

• 不同缩放级别需要不同细节程度• 缩放越小，要素顶点数应越少• 减少数据传输量和渲染负担• 避免小缩放级别下顶点过密• 多级LOD需要多级简化版本• 简化应保持形状的主要特征• 简化比例：zoom每降一级，顶点减半 典型场景：海岸线、道路、行政边界 目标：在可接受的误差下最大化简化

Douglas-Peucker算法

• 经典线要素简化算法，1973年提出• 算法思想：递归分割，保留关键点• 算法步骤： 1. 连接曲线首尾两点成直线 2. 找出曲线上距离直线最远的点 3. 距离>阈值：保留该点，递归处理两段 4. 距离≤阈值：舍去中间所有点 5. 递归结束，输出简化后的点序列 时间复杂度：O(n log n) 平均，$O(n^2)$ 最坏 阈值选择：与缩放级别成正比

渲染中的空间索引——加速可见性查询

为什么渲染需要空间索引

• 视锥体剔除需要快速查询可见要素• 每帧都要做可见性判断• 要素数量大，线性扫描不可行• 空间索引将O(n)降为O(log n)• 瓦片本身就是一种空间索引（规则网格）• 瓦片内要素多了仍需进一步索引• 动态要素更需要空间索引加速 应用场景：点要素碰撞检测、视口要素查询 静态要素可预构建索引，动态需实时更新

常用空间索引结构

• R树：最经典的空间索引，适合矩形范围查询• R*树：R树优化版，分裂策略更优• 四叉树：二维空间四叉划分，实现简单• k-d树：k维二叉搜索树，适合点数据• 网格索引：规则网格划分，最简单• STR树：批量加载优化的R树变体• 选择策略：静态数据多→R树/STR树 动态数据多→四叉树/网格索引 点数据→k-d树

GPU渲染管线与着色器基础

GPU渲染管线

• 顶点输入→顶点着色器→图元装配→光栅化→片元着色器→测试混合• 顶点着色器（VS）：坐标变换、顶点属性计算• 片元着色器（FS）：像素颜色计算、纹理采样• VBO（顶点缓冲对象）：GPU端存储顶点数据• VAO（顶点数组对象）：顶点属性配置• EBO（索引缓冲对象）：三角形索引存储• 纹理（Texture）：图片数据上传到GPU 并行特性：GPU数千核心并行处理顶点/片元 性能关键：尽量减少CPU-GPU数据传输

地图渲染中的着色器应用

• 线渲染：几何着色器生成线的条带• 虚线/点线：片元着色器中discard实现• 线帽/线接：几何着色器处理端点和交点• SDF文字：片元着色器计算距离场• 地形着色：高度→颜色映射• 样式属性：顶点属性传递样式参数• 后处理：模糊、发光、抗锯齿等效果 性能优化：减少shader切换，合并绘制 调试工具：RenderDoc、Spector.js、WebGL Inspector

3D地图渲染基础

3D地图渲染技术

- 2D→2.5D→3D的演进路径
- 2.5D：假3D，透视投影下的2D渲染
- 真3D：三维模型、真实光照、阴影
- 建筑挤出（Extrusion）：最基础的3D建筑 根据建筑高度将2D轮廓向上挤出 侧面和顶面分别赋予不同材质
- 倾斜摄影：真实照片重建的3D模型
- BIM模型：精细的建筑信息模型

城市级3D：百万级建筑模型的高效渲染 技术挑战：数据量巨大，需要LOD和实例化

3D渲染关键技术

- 相机系统：透视投影、FOV、近远裁剪面
- 光照系统：方向光、点光、环境光
- 材质系统：PBR（基于物理的渲染）
- 阴影技术：阴影贴图（Shadow Map）
- 天空盒：环境贴图模拟天空
- 雾效：大气散射、距离雾
- 地形渲染：DEM网格 + 纹理贴图

性能优化：视锥体剔除 + 背面剔除 + LOD

进阶：GPU Instancing、GPU Culling

未来趋势

地图渲染引擎的三大发展趋势

1

3D化与实景化 从平面地图到数字孪生

从2D矢量地图→2.5D倾斜→3D城市→实景三维→数字孪生，地图的维度和真实感不断提升。倾斜摄影、激光点云、AI三维重建推动3D数据成本下降，浏览器端WebGPU和移动端GPU性能提升让3D渲染成为可能。未来的地图将是'所见即所得'的真实世界数字镜像。

2

AI辅助渲染 智能优化与内容生成

AI正在改变地图渲染的方方面面：智能标注布局（学习人类制图师的审美）、AI纹理生成（自动生成建筑立面纹理）、智能LOD调度（预测用户行为提前加载）、超分辨率渲染（AI放大低分辨率数据）、风格迁移（一键切换地图艺术风格）。AI将成为渲染引擎的'智能大脑'。

3

实时渲染与云渲染 边界逐渐模糊

5G+云计算推动云渲染发展，地图渲染可以在云端完成后以视频流推送到终端。终端不再需要强大GPU，任何设备都可以获得顶级渲染质量。同时，实时数据（交通、天气、事件）的动态渲染要求越来越高，地图从'静态图片'变成'实时信息流'。



知识图谱

本节课知识点全景

架构与管线

五层架构、渲染管线五阶段、跨平台抽象层

数据与坐标

MVT矢量瓦片、瓦片金字塔、墨卡托投影、坐标变换

样式系统

Mapbox样式规范、表达式引擎、运行时样式切换

文字标注

点线面标注、碰撞检测、SDF/MSDF渲染

接口定义

G8渲染组与其他组的协作接口

G8 ↔ G7 导航引擎组

• 输入（G7→G8）： 当前位置与朝向（经纬度、航向角） 导航状态（巡航/转弯/到达等） 路径几何（引导线、转向箭头） 实时路况信息（拥堵、事故）
• 输出（G8→G7）： 地图视口状态（中心、缩放、旋转） 用户交互事件（点击、长按、选择） 渲染完成回调（首屏、加载状态）

G8 ↔ G6/G10 接口

• G8 ↔ G6 定位匹配组： 输入：定位点（经纬度+精度） 输出：地图上的定位蓝点渲染 提供：地图坐标↔屏幕坐标转换API
• G8 ↔ G10 语音交互组： 输入：语音指令（放大/缩小/搜周边） 输出：地图操作执行结果反馈 提供：地图控件状态查询接口 支持：语音引导的界面高亮和动画

本章小结

本次课核心判断

🎯 核心判断

地图渲染引擎的核心是'在有限资源下，把海量地理数据以最美观、最高效的方式呈现给用户'——数据是基础，算法是核心，性能是生命线。

数据层

矢量瓦片MVT格式、瓦片金字塔LOD调度、Web墨卡托投影与坐标变换链，是渲染引擎的数据基石

渲染层

五层架构、渲染管线五阶段、GPU加速渲染、Draw Call优化、LRU缓存、视锥体剔除，性能优化永无止境

样式与标注

Mapbox样式规范、表达式求值引擎、运行时样式切换、SDF文字渲染、碰撞检测与标注放置，决定地图的美观度和可读性

交互与扩展

惯性滚动、捏合缩放、相机控制、无人机/AUV特种可视化、3D地图、AI辅助渲染，应用场景不断拓展

课堂讨论

如果让你从零设计一个地图渲染引擎，你会先做什么？后做什么？

观点1

先做矢量瓦片解析和基本线面渲染——能把数据画出来是第一步，先跑通主链路再谈优化和美观

观点2

先做样式系统和标注系统——地图好不好看是用户第一感受，样式和标注是地图渲染的灵魂

观点3

先做性能优化框架——一开始就考虑性能，不然后期重构代价太大，尤其是渲染引擎这种底层系统

观点4

先做跨平台抽象层——一套代码多端运行是现代引擎的标配，先把架构搭好再填功能

💡 **讨论引导**

没有标准答案。实际工程中，往往是'最小可用原型→逐步迭代'的思路：先跑通数据→渲染→显示主链路，再逐步完善样式、标注、性能、交互。关键是保持架构的可扩展性。

常见问题

FAQ — 学生最常问的问题

Q: 矢量瓦片和栅格瓦片怎么选?

A: 新项目优先选矢量瓦片：体积小、样式灵活、支持交互、缩放无损。只有影像/地形等光栅数据用栅格瓦片。如果是简单展示、对交互要求低，栅格瓦片实现更简单。

Q: 为什么地图文字不会重叠？原理是什么?

A: 核心是碰撞检测算法。每个标注有一个包围盒，按优先级排序后逐个放置，放置前检测是否与已放置的标注碰撞，有碰撞就跳过或换位置。常用空间索引（网格、四叉树）加速检测。

Q: SDF文字和位图文字有什么区别?

A: 位图文字每个字号一套纹理，缩放有锯齿，但小字质量好。SDF文字用距离场存储，单张纹理支持任意缩放，还能做描边阴影效果，但极小字质量略逊。现代地图引擎基本都用SDF/MSDF。

Q: 视锥体剔除为什么能提升性能?

A: 视锥体剔除在CPU端判断哪些物体在相机视野内，只把可见的物体提交给GPU渲染。地图中90%以上的瓦片和要素在视口外，剔除后Draw Call和三角形数量大幅减少，性能提升非常明显。

Q: Mapbox GL和MapLibre GL是什么关系?

A: MapLibre GL是Mapbox GL的社区开源分支。Mapbox在2020年将GL JS v2改为非开源协议，社区便fork了v1版本继续维护，形成了MapLibre项目。两者API基本兼容，MapLibre完全开源免费。

Q: 学习地图渲染应该从哪里入手?

A: 建议路径：1）理解矢量瓦片和样式规范；2）用MapLibre GL JS做一个demo；3）阅读MapLibre源码（Style→Source→Tile→Painter）；4）动手实现一个简化版渲染器（Canvas即可）；5）深入WebGL和GPU渲染。

参考文献与延伸阅读

技术规范与标准

• Mapbox Vector Tile Specification v2.1• Mapbox Style Specification v8• OGC Vector Tiles Pilot• Web Mercator Projection (EPSG:3857)• Protocol Buffers (protobuf) 官方文档

开源项目与工具

• MapLibre GL JS / MapLibre Native• OpenLayers - 开源GIS Web库• Leaflet - 轻量Web地图库• Mapnik - 服务端地图渲染引擎• msdfgen - MSDF字体生成工具• earcut - 高效三角剖分库

推荐书目与论文

• 《Real-Time Rendering 4th》- 实时渲染经典• 《WebGL Programming Guide》• 《Fundamentals of Computer Graphics》• Douglas & Peucker (1973) - 线要素简化经典论文• SDF字体：Green (2007) - Improved Alpha-Tested Magnification• Chazelle (1991) - 线性时间三角剖分算法

CHAPTER · 核心问题

理论已经掌握，接下来是实战环节

- G8渲染组：构建一个可运行的地图渲染引擎原型
- 目标：从瓦片加载到屏幕渲染，跑通完整链路
- 其他组：理解渲染层接口，做好联调准备

任务1与任务2——数据加载与基础渲染

任务1

瓦片加载与解析

核心功能：瓦片坐标计算（经纬度↔瓦片坐标）、MVT PBF格式解析（推荐使用pbf库）、瓦片网络请求与缓存（内存LRU+localStorage）、视口瓦片列表计算。
输出：Tile类、TileLoader类、要素数据结构（GeoJSON-like）。
验收：能正确加载并解析指定z/x/y的瓦片，打印要素数量和类型统计。

任务2

基本图元渲染

核心功能：Canvas 2D渲染上下文封装、坐标变换矩阵（平移+缩放+旋转）、点要素渲染（圆点/图标）、线要素渲染（折线、线宽、颜色）、面要素渲染（多边形填充+边框）、视口动态更新（拖动时加载/卸载瓦片）。
输出：Renderer类、Layer类、Transform类。
验收：拖动地图时能流畅显示道路面等基本要素，帧率>30fps。

任务3与任务4——样式系统与文字标注

任务3

样式系统实现

核心功能：JSON样式配置解析、样式图层与数据图层映射、属性过滤（filter）、基础样式属性（颜色/线宽/透明度/半径）、简单表达式支持（get、literal、插值）、运行时样式切换（日间/夜间主题）。
输出：Style类、Expression类、ThemeManager。
验收：加载样式文件后地图按样式渲染，切换主题时平滑过渡，所有图层正确应用样式。

任务4

文字标注与碰撞

核心功能：点标注渲染（文字+图标组合）、标注包围盒计算、碰撞检测算法（AABB检测+空间网格加速）、标注优先级排序、渐进式标注（缩放时逐步显示）、标注稳定性优化（位置缓存）。
输出：LabelLayer类、CollisionDetector类。
验收：缩放过程中标注无明显重叠，标注数量随缩放级别合理变化，无剧烈抖动。

任务5——交互系统与性能优化

任务5

交互与性能优化（8h）

核心功能：鼠标/触摸事件处理（拖拽平移、滚轮/双指缩放、双击放大）、惯性滚动实现（速度计算+指数衰减+边界回弹）、视锥体剔除（只渲染视口内瓦片和要素）、LRU瓦片缓存（内存限制+淘汰策略）、性能监控（FPS统计、渲染耗时统计）。
输出：Interaction类、Camera类、Cache类、Performance类。
验收：地图操作流畅无卡顿，惯性滚动自然，缩放跟手，帧率稳定在30fps以上，内存使用可控。

CHAPTER · 核心问题

渲染引擎是地图的'画笔'——算法决定能画多快，审美决定能画多美

第9次课 · 地图渲染与可视化引擎

G8组的同学们，用你们的代码画出最棒的地图吧！

下节课：第10次课 语音交互与多模态引导